

Luồng dữ liệu: Unidirectional Data Flow – **Luồng dữ liệu 1 chiều**

*Data truyền từ State -> Widget để render giao diện -> Sự kiện phát sinh đi lên do User tương tác -> gửi Event lên Controller -> thay đổi State

*Cách quản lý trạng thái State: Cách dùng dữ liệu 1 chiều xuyên suốt app như các mô hình: Provider, ValueNotifier,... giúp cho không phải truyền biến thủ công qua nhiều tầng widget con

=====

Widget Lifecycle:

createState: tạo State và gắn với Widget

-> **initState** (chạy duy nhất 1 lần, dùng để khởi tạo controller, hoặc gọi API lần đầu

-> **didChangeDependencies:** kích hoạt ngay sau khi initState hoặc mỗi khi đối tượng thay đổi giá trị

-> **build:** chạy nhiều lần để mô tả cấu trúc giao diện (lưu ý: không nên xử lý không đồng bộ ở đây)

-> **didUpdateWidget:** được gọi khi Widget cha rebuild và cấu hình widget con nhưng vẫn giữ nguyên key

-> **setState:** báo cho Flutter Framework biết dữ liệu nội bộ đã thay đổi -> yêu cầu build lại

-> **dispose:** chạy khi đối tượng State bị xóa vĩnh viễn. Bắt buộc dùng để hủy: TextEditingController

AnimationController

StreamController

để tránh rò rỉ bộ nhớ (Memory leak: xảy ra khi 1 đối tượng không sử dụng nhưng vẫn bị tham chiếu -> thông thường quên đóng Stream, quên không hủy Timer, sử dụng context tĩnh.

Chú ý: Về mặt quản lý bộ nhớ, Dart VM phân bổ và kiểm soát tài nguyên

*Garbage Collection: gom Rác bộ nhớ -> thu gom các đối tượng sử dụng ngắn hạn (ví dụ: các widget được tạo mới liên tục khi rebuild) -> dọn dẹp các đối tượng đã có từ lâu nhưng không sử dụng (service, controller)

=====

Tại sao flutter nhanh???

Vì nó sử dụng kiến trúc nền tảng 3 cây

Widget-> (Bản thiết kế, bất biến): Tập hợp các cấu hình giao diện nhẹ, bất biến, được khởi tạo và hủy liên tục mỗi khi rebuild mà không tốn tài nguyên.

|

Element -> (bộ điều phối, cầu nối, lifecycle): cầu nối lâu dài giữa Widget và RenderObject để giữ trạng thái và quản lý vòng đời. Khi widget rebuild, element sẽ so sánh xem widget mới có cùng key không??? Nếu trùng key thì chỉ cập nhật cấu hình cho RenderObject mà không vẽ lại từ đầu. Nếu khác key thì hủy element cũ và tạo element mới

|

RenderObject-> vẽ layout thực tế

=====

Kiến trúc Flutter: tách biệt các tầng

PresentationLayer (UI, State)

DomainLayer (UseCases, Entities)

DataLayer (Repository, Model, DataSource)

Áp dụng trong code:

-Nên sử dụng từ khóa **const** trước constructor để Flutter tái sử dụng instance và bỏ qua bước rebuild

-Không nên gọi tác vụ nặng trong hàm build (nên đưa vào worker qua **compute**)

-Tái sử dụng danh sách dài bằng **Listview.builder** thay vì nạp toàn bộ danh sách